WEEK 3:

1. Aim

To establish connectivity between Python and MongoDB and develop an LLM monitoring system that records API usage, token consumption, response time, estimated cost and request status.

2. Software Requirements

- Python
- Visual Studio Code
- MongoDB Community Server
- MongoDB Compass
- Google Gemini API Key
- Ollama
- Llama 3.2 Model

3. Libraries Used

1. **PyMongo:** Used to connect Python applications with MongoDB.
2. **Google GenAI:** Used to communicate with the Gemini API.
3. **Requests:** Used to send HTTP requests to Ollama.
4. **Python-dotenv:** Used to load environment variables from the .env file.
5. **Datetime:** Used to record timestamps.
6. **Time:** Used to calculate response latency.

4. Installation and Setup

Step 1: Install MongoDB

Download and install:

- MongoDB Community Server
- MongoDB Compass

Step 2: Check MongoDB Connectivity

Open PowerShell and execute:

```
Get-Service MongoDB
```

If the service is not running, execute:

```
Start-Service MongoDB
```

Step 3: Create Database

Open MongoDB Compass and create:

Database Name: `llm_monitoring`

Collection Name: `call_logs`

Step 4: Download Ollama Model

Execute the following commands:

```
ollama pull llama3.2
```

To run the model:

```
ollama run llama3.2
```

5. Project Structure

Create a folder named `Week-3` in Visual Studio Code.

Create the following files:

1. cost_calculator.py
2. llm_client.py
3. main.py
4. mongo_logger.py
5. requirements.txt
6. .env

6. API Key Configuration

Create a `.env` file and add:

```
GEMINI_API_KEY=your_actual_api_key
```

7. Python Programs

A. cost_calculator.py

This file calculates the estimated cost based on input and output token consumption.

```
PRICING = {
    "gemini-3-flash-pro": {
        "input": 0.50,
        "output": 2.00
    }
}

def calculate_cost(provider, model, input_tokens, output_tokens):

    if provider == "OllamaProvider":
        return 0.0

    rates = PRICING.get(model)

    if rates is None:
        return None

    input_cost = (
        input_tokens * rates["input"]
    ) / 1_000_000

    output_cost = (
        output_tokens * rates["output"]
    ) / 1_000_000

    total_cost = input_cost + output_cost
```

```
    return round(total_cost, 10)
```

Note: The pricing values are sample values from the lab manual and must not be treated as verified current API prices.

B. llm_client.py

This file implements the common LLM interface, Gemini provider, Ollama provider, monitoring and retry mechanisms.

[Insert the complete llm_client.py code from the Week 3 lab manual here.]

C. mongo_logger.py

This file establishes the MongoDB connection and stores LLM call details.

```
from pymongo import MongoClient
from datetime import datetime, timezone

class MongoLogger:

    def __init__(self):

        self.client = MongoClient(
            "mongodb://localhost:27017/"
        )

        self.db = self.client["llm_monitoring"]
        self.collection = self.db["call_logs"]

    def log_call(self, log_data):

        log_data["timestamp"] = datetime.now(timezone.utc)

        self.collection.insert_one(log_data)

        print("Call details saved in MongoDB!")

    def close(self):
```

```
        self.client.close()
```

D. main.py

This is the main program used to select the LLM provider and generate responses.

```
# main.py
import os
from dotenv import load_dotenv
from llm_client import (
    GeminiProvider,
    OllamaProvider,
    LLMClient
)
from mongo_logger import MongoLogger
# Load environment variables
load_dotenv(override=True)
print("================================")
print("     WEEK 3: LLM MONITORING")
print("================================")
print("1. Gemini")
print("2. Ollama")
choice = input("\nSelect provider (1/2): ").strip()
logger = None
try:
    # Connect to MongoDB
    logger = MongoLogger()
    # Select AI provider
    if choice == "1":
        api_key = os.getenv("GEMINI_API_KEY")
        if not api_key:
            raise ValueError("Gemini API key not found in .env")
        provider = GeminiProvider(api_key)
    elif choice == "2":
        provider = OllamaProvider("llama3.2")
    else:
        raise ValueError("Invalid selection. Choose 1 or 2.")
    # Create common client
    client = LLMClient(provider, logger)
    # Get user prompt
    prompt = input("\nEnter your question: ")
```

```
    # Generate response
    answer = client.generate(prompt)
    print("\n========== AI RESPONSE ==========")
    print(answer)
    print("\n========== COMPLETED ==========")
except Exception as error:
    print("\nApplication error:", error)
finally:
    if logger is not None:
        logger.close()
```

E. requirements.txt

```
pytest
google-genai
python-dotenv
pymongo
```

8. Execution

Step 1: Install Required Libraries

Open the VS Code terminal and execute:

```
pip install -r requirements.txt
```

Step 2: Start MongoDB

Ensure that the MongoDB service is running.

Step 3: Start Ollama

Open Ollama and ensure the Llama 3.2 model is available.

Step 4: Run the Python Program

Execute:

```
python main.py
```

Step 5: Select the Provider

The program displays:

1. Gemini
2. Ollama

Select either option.

Step 6: Enter a Prompt

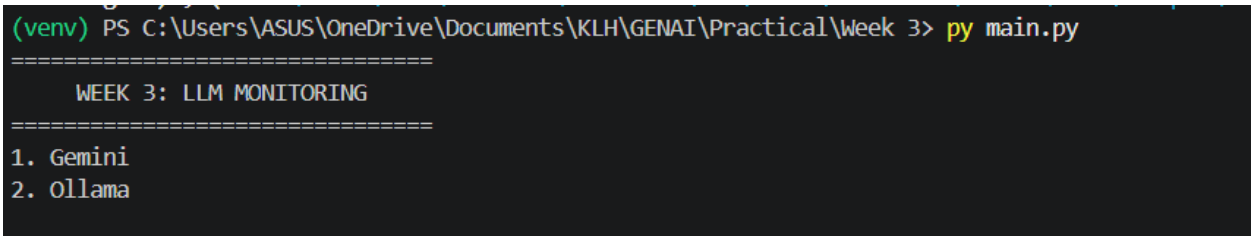
Example:

```
What is Artificial Intelligence?
```

The selected LLM generates a response.

9. Output

Output 1: Provider Selection



Output 2: AI Response

```
(venv) PS C:\Users\ASUS\OneDrive\Documents\KLH\GENAI\Practical\Week 3> py main.py
==============================
     WEEK 3: LLM MONITORING
==============================
1. Gemini
2. Ollama

Select provider (1/2): 2

Enter your question: What is AI
Call details saved in MongoDB!

========== AI RESPONSE ==========
AI, or Artificial Intelligence, refers to the simulation of human intelligence in machines that are programmed to thin
k like humans and mimic their actions. The term can also be applied to any machine that exhibits traits associated wit
h a human mind such as learning and problem-solving.

There are several types of AI, including:

1. **Narrow or Weak AI**: This type of AI is designed to perform a specific task, such as facial recognition, language
 translation, or playing chess.
2. **General or Strong AI**: This type of AI is designed to perform any intellectual task that a human can, and is con
sidered the holy grail of AI research.
3. **Superintelligence**: This type of AI is significantly more intelligent than the best human minds, and has the pot
ential to solve complex problems that are currently unsolvable.
```

Output 3: MongoDB Logging

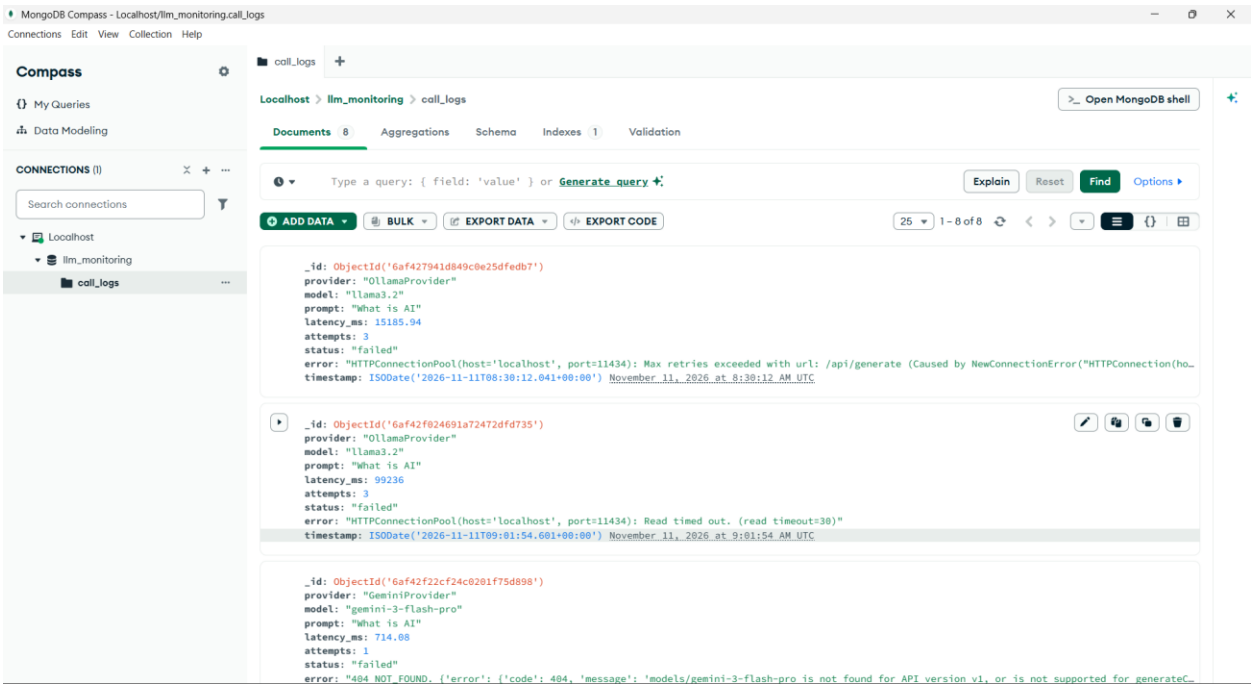
```
==============================
     WEEK 3: LLM MONITORING
==============================
1. Gemini
2. Ollama

Select provider (1/2): 1

Enter your question: What is AI
Call details saved in MongoDB!

Application error: 504 DEADLINE_EXCEEDED. {'error': {'code': 504, 'message': 'Deadline expired before operation could
complete.', 'status': 'DEADLINE_EXCEEDED'}}
```

Output 4: MongoDB Compass



Output 5: Stored Monitoring Details



10. Observations

1. Successfully installed MongoDB Community Server and MongoDB Compass.
2. Established connectivity between Python and MongoDB.

3. Created the `llm_monitoring` database and `call_logs` collection.
4. Integrated Gemini and Ollama as LLM providers.
5. Observed that the application records input and output token consumption.
6. Implemented cost calculation based on token usage.
7. Recorded response latency to monitor the performance of LLM requests.
8. Implemented retry mechanisms to handle temporary failures.
9. Successfully stored LLM request details in MongoDB.
10. Observed the stored records using MongoDB Compass.

11. Result

Successfully established MongoDB connectivity and implemented an LLM monitoring system using Python. The system tracks LLM usage, token consumption, estimated cost, response latency and request status and stores the information in MongoDB.